



نام پروژه: **codiva**

اعضای گروه: فائزه دلیلی، ریحانه مهدوی کیا، مائده اسلامی، ریحانه سرابی

استاد: مهندس رشید باقری

درس توسعه نرم افزار

تکنولوژی های اصلی: **React ، JavaScript ، JSON**

نوع پروژه: فرانت اند (**FrontEnd**)

فهرست مطالب

بخش اول: معماری و ساختار سیستم

1	مقدمه و الگوی کلی معماری
2	الگوی معماری پروژه و ویژگی ها
2	پشته تکنولوژی
3	فناوری های اصلی و لایه بندی سیستم.....
4	نمودار جریان داده.
5	امنیت و پایداری.
7	ساختار درختی پوشه ها.....
9	بخش دوم:فرآیند های PSPEC
15	بخش سوم:مستندسازی کامپوننت های اصلی
17	بخش چهارم:ساختار پایگاه داده
17	ساختار داده کاربران
19	بخش پنجم:مدیریت شاخه بندی پروژه
20	خلاصه تاریخچه اسپرینت ها
21	بخش ششم:مستندات هویت بصری سیستم
24	بخش هفتم:نمودار های UML
24	نمودار usecase
26	نمودارهای sequence
32	بخش هشتم:اجرای پروژه و چشم انداز توسعه.....

بخش اول: معماری و ساختار سیستم

در این بخش به معرفی ساختار کلی پروژه، نحوه سازمان‌دهی اجزای نرم‌افزار، فناوری‌های مورد استفاده، جریان داده‌ها، ملاحظات امنیتی و پایداری، و در نهایت ساختار پوشه‌های پروژه می‌پردازیم.

مقدمه

پروژه Codiva یک سیستم مدیریت حضور و غیاب مبتنی بر وب است که با استفاده از React پیاده‌سازی شده و برای شبیه‌سازی Backend از JSON Server / Fake Server بهره می‌برد.

هدف اصلی این سامانه، مدیریت اطلاعات کاربران، ثبت و نمایش رکوردهای حضور و غیاب، کنترل دسترسی بر اساس نقش کاربر، و فراهم کردن امکاناتی مانند جستجو، فیلتر، مرتب‌سازی و گزارش‌گیری است.

در این پروژه دو نقش اصلی تعریف شده است:

- Admin: با دسترسی کامل به مدیریت کاربران و مشاهده همه رکوردها

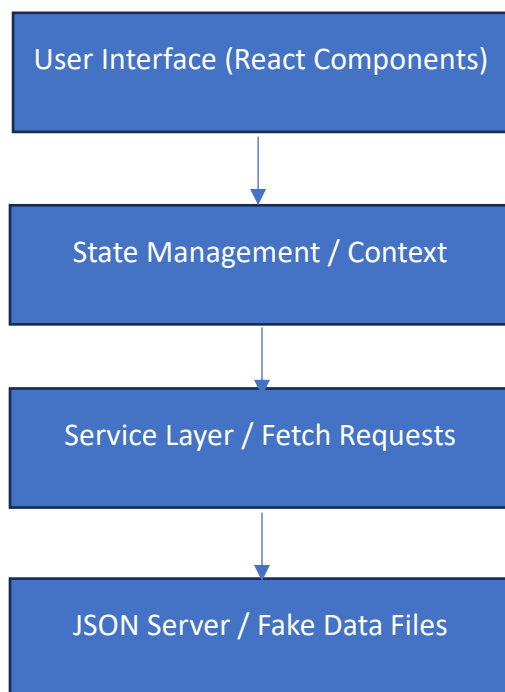
- User: با دسترسی محدود به اطلاعات و گزارش‌های شخصی خود

این پروژه با تکیه بر معماری کامپوننت‌محور React، مدیریت وضعیت، اعتبارسنجی فرم‌ها و تفکیک لایه‌های منطقی، به گونه‌ای طراحی شده که هم قابل نگهداری باشد و هم قابل توسعه.

الگوی کلی معماری

معماری کلی سیستم بر پایه یک ساختار Frontend-Driven است؛ یعنی منطق اصلی برنامه در سمت کاربر پیاده‌سازی شده و ارتباط با داده‌ها از طریق درخواست‌های HTTP و فایل‌های JSON انجام می‌شود.

الگوی معماری پروژه:



ویژگی‌های این معماری

- ✓ کامپوننت‌محور بودن رابط کاربری
- ✓ جداسازی مسئولیت‌ها
- ✓ مقیاس‌پذیری مناسب
- ✓ سادگی در نگهداری و توسعه
- ✓ قابلیت مدیریت نقش‌ها و سطح دسترسی

پشته تکنولوژی

پشته فناوری پروژه از ابزارها و کتابخانه‌هایی تشکیل شده که هرکدام نقش مشخصی در توسعه سیستم دارند.

فناوری های اصلی:

فناوری	نقش
React	ساخت رابط کاربری و کامپوننت‌ها
TypeScript	افزایش ایمنی نوع داده و کاهش خطا
Tailwind css	طراحی سریع و واکنش گرا
React Router	مدیریت مسیرها و صفحات
React Hook Form	مدیریت فرم ها
Yup	اعتبارسنجی فرم ها
Context API	مدیریت state سراسری
JSON server	شبیه سازی API و ذخیره داده
Fetch API	ارسال و دریافت داده
LocalStorage	ذخیره موقت اطلاعات کاربر

این ترکیب فناوری‌ها باعث شده سیستم هم از نظر رابط کاربری مدرن باشد و هم از نظر منطق داده، ساختاری منسجم و قابل کنترل داشته باشد.

لایه بندی سیستم

سیستم به صورت منطقی به چند لایه تقسیم می شود تا هر بخش وظیفه مشخصی داشته باشد.

1. لایه نمایش (Presentation Layer)

این لایه شامل تمام صفحات و کامپوننت‌های رابط کاربری است، مانند:

- Login Page
- Dashboard
- Users Management
- Create User
- Edit User
- Report Modal

وظیفه این لایه، نمایش اطلاعات و دریافت ورودی از کاربر است.

2. لایه مدیریت وضعیت (State Management Layer)

در این لایه، اطلاعات سراسری برنامه نگهداری می‌شود؛ مانند:

- اطلاعات کاربر وارد شده
- لیست کاربران
- رکوردهای حضور و غیاب
- وضعیت باز و بسته بودن مودال‌ها

3. لایه سرویس‌ها (Service Layer)

این لایه مسئول ارتباط با داده‌ها و ارسال درخواست‌هاست.

به جای Backend واقعی، از fetch و JSON Server استفاده می‌شود.

مثال:

```
fetch("http://localhost:3001/users")
```

4. لایه کنترل دسترسی (Access Control Layer)

این لایه بررسی می‌کند که کاربر چه نقشی دارد و به کدام صفحه یا داده می‌تواند دسترسی داشته باشد.

```
currentUser.personalInfo.isAdmin
```

✓ اگر true باشد → Admin

✓ اگر false باشد → User

نمودار جریان داده

نمودار جریان داده نشان می‌دهد که اطلاعات چگونه بین کاربر، رابط کاربری، سرویس‌ها و منبع داده جابه‌جا می‌شوند.

1. جریان داده در Login

1. کاربر نام کاربری و رمز عبور را وارد می کند.
2. اطلاعات به فرم Login ارسال می شود.
3. داده ها از users.json خوانده می شوند.
4. اعتبار اطلاعات بررسی می شود.
5. در صورت موفقیت، نقش کاربر تعیین می شود.
6. کاربر به Dashboard هدایت می شود.

2. جریان داده در Dashboard

1. کاربر وارد داشبورد می شود.
2. اطلاعات کاربر جاری از Context یا localStorage خوانده می شود.
3. رکوردهای حضور و غیاب از timeX.json دریافت می شوند.
4. داده ها بر اساس تاریخ و نقش فیلتر می شوند.
5. جدول داشبورد نمایش داده می شود.

3. جریان داده در Users Management

1. Admin وارد صفحه مدیریت کاربران می شود.
2. لیست کاربران از JSON Server دریافت می شود.
3. عملیات جستجو، فیلتر، مرتب سازی و CRUD روی داده ها انجام می شود.
4. داده های جدید دوباره به سرور ارسال یا به روزرسانی می شوند.

امنیت و پایداری

1. امنیت

امنیت این سیستم بر پایه Role-Based Access Control طراحی شده است.

اقدامات امنیتی اصلی:

- بررسی نقش کاربر با `isAdmin`
- محدودسازی دسترسی به صفحات مدیریتی
- جلوگیری از ورود کاربران عادی به بخش `Users`
- اعتبارسنجی فرمها با `Yup`
- کنترل ورودیهای نامعتبر
- جلوگیری از ثبت داده ناقص یا اشتباه

نمونه کنترل دسترسی:

```
if (!currentUser.personalInfo.isAdmin){  
  
  redirectTo("/dashboard");  
  
}
```

2. پایداری

پایداری سیستم با استفاده از چند اصل مهم تقویت شده است:

عوامل مؤثر بر پایداری:

- استفاده از **TypeScript** برای کاهش خطاهای نوع داده
- استفاده از **React Hook Form** برای مدیریت فرمها به صورت پایدار و سبک
- استفاده از **Yup** برای جلوگیری از ثبت داده نامعتبر
- تفکیک مسئولیتها بین صفحه، سرویس و `Context`
- استفاده از **Fake Server** برای دادههای قابل پیشبینی
- ساختار ماژولار و قابل نگهداری

این ساختار باعث می شود سیستم در برابر خطاهای رایج، تغییرات آینده و توسعه های بعدی، پایدارتر و قابل اعتمادتر باشد.

ساختار درختی پوشه‌ها (folder structure)

ساختار پروژه به صورت ماژولار سازمان‌دهی شده تا هر بخش وظیفه مشخصی داشته باشد.



assets/	نگهداری منابع ثابت پروژه شامل تصاویر لوگو (logo.png) و استایل‌های عمومی (App.css)
components/common/	کامپوننت‌های پایه و قابل استفاده مجدد (Button, Input, Select) برای یکپارچگی طراحی UI.
components/layout/	مدیریت ساختار کلی صفحه (DashboardLayout, Desktop/Mobile Layouts) برای پاسخ‌گویی واکنش‌گرا.
components/users/	کامپوننت‌های اختصاصی مدیریت کاربران (Sorting, UserList) و جداول نمایش گزارش‌ها.
components/modal/	مدیریت نمایش پنجره‌های بازشو و گزارش‌های جزئی کاربران (ReportModal).
constant/	فایل‌های تنظیمات ثابت (مانند ستون‌های جداول LogCol, ReportCol, tableCol) و داده‌های خام (users.json, timex.json).
context/	مدیریت وضعیت سراسری (State Management) برای احراز هویت (Auth.Context) و اطلاعات کاربران.
core/	لایه اصلی زیرساختی شامل اینترفیس‌ها (UserInterface), مدل‌های داده (UserTable.type) و توابع کمکی ذخیره‌سازی (storage.ts).
pages/	صفحات اصلی و مسیرهای برنامه (Login, Dashboard, Users, CreateUser, EditUser) که منطق اصلی هر صفحه در آن است.
container/	مدیریت کانتینرهای اصلی و نقطه شروع رندرینگ برنامه (main.tsx).

بخش دوم: فرآیندهای PSPEC

فرآیند 1 نام فرآیند: Authentication / Login Process هدف: بررسی اطلاعات ورود کاربر و تعیین نقش او در سیستم
ورودی ها: • نام کاربری • رمز عبور
منبع داده: users.json
خروجی ها: • اطلاعات کاربر لاگین شده • Admin یا user • پیام خطا در صورت نامعتبر بودن ورود
منطق پردازش: 1. دریافت username و password از فرم Login 2. خواندن لیست کاربران از Fake Server 3. جستجوی کاربر با username و password یکسان 4. اگر کاربر پیدا نشود، نمایش خطای Login 5. اگر کاربر پیدا شود، ذخیره اطلاعات کاربر 6. بررسی مقدار personalInfo.isAdmin 7. هدایت کاربر به Dashboard
قوانین: ✓ فقط کاربران موجود در users.json امکان ورود دارند. ✓ نقش کاربر از طریق personalInfo.isAdmin تعیین می شود. ✓ پس از ورود موفق، همه کاربران وارد Dashboard می شوند. ✓ سطح دسترسی بعد از ورود بر اساس نقش کنترل می شود.

فرآیند 2

نام فرآیند: **Load Attendance Dashboard**

هدف: نمایش رکوردهای حضور غیاب روز جاری بر اساس نقش کاربر

ورودی ها:

- کاربر لاگین شده
- لیست رکورد های حضور غیاب

منبع داده:

Timex.json

خروجی ها:

- رکوردهای قابل نمایش در داشبورد

منطق پردازش:

1. دریافت کاربر لاگین شده از **Context** یا **localStorage**
2. دریافت رکوردهای حضور و غیاب از **timeX.json**
3. استخراج تاریخ جاری
4. فیلتر رکوردها بر اساس تاریخ جاری
5. اگر کاربر **Admin** باشد، تمام رکوردهای امروز نمایش داده می شوند
6. اگر کاربر **User** باشد، فقط رکوردهای مربوط به خودش نمایش داده می شود
7. داده ها در جدول **Dashboard** نمایش داده می شوند

قوانین:

- ✓ **Dashboard** برای **Admin** و **User** قابل دسترسی است .
- ✓ **Admin** تمام رکوردهای امروز را می بیند .
- ✓ **User** فقط رکوردهای خودش را می بیند .
- ✓ رکوردهای گذشته فقط از طریق **Report** نمایش داده می شوند.

فرآیند 3

نام فرآیند: Authorization / Route Protection

هدف: جلوگیری از دسترسی کاربران عادی به صفحات غیرمجاز

ورودی ها:

- کاربر لاگین شده
- مسیر درخواستی

خروجی ها:

- اجازه ورود به صفحه
- مسیر جایگزین در صورت عدم دسترسی

منطق پردازش:

1. دریافت مسیر درخواستی
2. بررسی وجود کاربر لاگین شده
3. اگر کاربر لاگین نشده باشد، انتقال به Login
4. اگر مسیر مربوط به Users Management باشد :
 - بررسی مقدار `personallInfo.isAdmin`
 - اگر مقدار `true` باشد، اجازه دسترسی داده می شود
 - اگر مقدار `false` باشد، انتقال به Dashboard
5. برای Dashboard ، هر دو نقش اجازه دسترسی دارند

قوانین:

- ✓ صفحه Users فقط برای Admin است.
- ✓ User عادی حتی با وارد کردن مستقیم URL نباید وارد صفحه Users شود.
- ✓ کاربر لاگین نشده نباید به صفحات داخلی سیستم دسترسی داشته باشد.

فرآیند 4

نام فرآیند: View Attendance Report

هدف: نمایش تاریخچه کامل حضور و غیاب یک کاربر در Modal

ورودی ها:

- شناسه کاربری که گزارش او انتخاب شده
- کاربر لاگین شده
- تمام رکوردهای حضور غیاب

خروجی ها:

- سوابق کامل حضور غیاب
- باز یا بسته بودن مودال

منطق پردازش:

1. کاربر روی دکمه Report کلیک می کند
2. شناسه کاربر انتخاب شده دریافت می شود
3. نقش کاربر جاری بررسی می شود
4. اگر Admin باشد، اجازه مشاهده گزارش داده می شود
5. اگر User باشد، بررسی می شود که `selectedUserId` با `currentUser.id` برابر باشد
6. تمام رکوردهای حضور و غیاب از `timeX.json` خوانده می شوند
7. رکوردها بر اساس `userId` فیلتر می شوند
8. Modal باز می شود و گزارش نمایش داده می شود

قوانین:

- ✓ Admin می تواند گزارش همه کاربران را ببیند .
- ✓ User فقط می تواند گزارش خودش را مشاهده کند .
- ✓ گزارش شامل تمام تاریخچه کاربر است، نه فقط تاریخ امروز.

فرآیند 5

نام فرآیند: Create New User

هدف: ایجاد کاربر توسط admin

ورودی ها:

- نام و نام خانوادگی
- نام کاربری
- ایمیل
- موقعیت شغلی
- سن و جنسیت
- رمز عبور

خروجی ها:

- کاربر ساخته شده
- پیام موفقیت
- خطاهای اعتبارسنجی

منطق پردازش:

1. Admin وارد صفحه Create User می شود
2. اطلاعات کاربر جدید را وارد می کند
3. فرم با React Hook Form مدیریت می شود
4. اعتبارسنجی با Yup انجام می شود
5. اگر داده ها معتبر نباشند، خطاها نمایش داده می شوند
6. اگر داده ها معتبر باشند، آبجکت newUser ساخته می شود
7. داده با درخواست POST به Fake Server ارسال می شود
8. Fake Server کاربر را ذخیره کرده و id تولید می کند
9. لیست کاربران در Context آپدیت می شود
10. پیام موفقیت نمایش داده می شود
11. فرم Reset می شود

قوانین:

- ✓ Admin فقط Admin امکان ایجاد کاربر دارد .
- ✓ تمام فیلدهای فرم اجباری هستند .
- ✓ ایمیل باید فرمت معتبر داشته باشد .
- ✓ رمز عبور باید حداقل ۶ کاراکتر باشد .
- ✓ کاربر جدید باید در فایل JSON/Fake Server ذخیره شود.

فرآیند 6 نام فرآیند: delete user هدف: حذف کاربر توسط admin
ورودی ها: • شناسه کاربر انتخاب شده
خروجی ها: • لیست کاربران بعد از حذف • پیام موفقیت حذف
منطق پردازش: 1. Admin روی دکمه Delete کلیک می کند 2. سیستم شناسه کاربر را دریافت می کند 3. درخواست حذف به Fake Server ارسال می شود 4. در صورت وجود کاربر ، ویژگی viewstate کاربر برابر با 0 می شود. 5. اگر حذف موفق باشد، لیست کاربران آپدیت می شود 6. کاربر از جدول حذف می شود
قوانین: ✓ فقط Admin امکان حذف کاربر دارد . ✓ User عادی نباید به عملیات حذف دسترسی داشته باشد . ✓ بهتر است قبل از حذف، پیام تأیید نمایش داده شود.

بخش سوم: مستندسازی فنی کامپوننت های اصلی

➤ Login page

وظیفه:

مدیریت ورود کاربران به سیستم.

مسئولیت ها

- دریافت username و password
- بررسی اطلاعات ورود
- تعیین نقش کاربر
- ذخیره کاربر جاری
- هدایت به Dashboard

➤ Dashboard page

وظیفه:

نمایش اطلاعات حضور و غیاب.

مسئولیت ها

- خواندن رکوردهای Attendance
- فیلتر بر اساس تاریخ جاری
- کنترل نمایش داده ها بر اساس نقش
- نمایش جدول
- اجرای Sorting و Filtering
- باز کردن Report Modal

➤ Users Management Page

وظیفه:

مدیریت کاربران توسط Admin.

مسئولیت‌ها

- نمایش لیست کاربران
- جستجو
- فیلتر
- مرتب‌سازی
- حذف کاربر
- رفتن به صفحه ایجاد یا ویرایش کاربر

➤ Report Modal

وظیفه:

نمایش تاریخچه کامل حضور و غیاب کاربر.

مسئولیت‌ها

- دریافت `selectedUserId`
- خواندن تمام رکوردهای `attendance`
- فیلتر بر اساس `userId`
- نمایش داده‌ها در `Modal`

بخش چهارم: مدیریت داده ها و ساختار پایگاه داده

در این پروژه به جای استفاده از یک پایگاه داده سنتی مانند MySQL یا MongoDB، از فایل های JSON به عنوان منبع داده استفاده شده است. این رویکرد برای پروژه های فرانت اند محور و نمونه های اولیه (Prototype) بسیار رایج است، زیرا بدون نیاز به راه اندازی یک سرور پایگاه داده پیچیده، امکان مدیریت و ذخیره اطلاعات را فراهم می کند.

برای شبیه سازی رفتار یک پایگاه داده واقعی، از ابزار **JSON Server (Fake Server)** استفاده شده است. این ابزار فایل های JSON را به یک **RESTful API** تبدیل می کند و امکان انجام عملیات های اصلی پایگاه داده (CRUD) شامل **ایجاد (Create)**، **خواندن (Read)**، **ویرایش (Update)** و **حذف (Delete)** را از طریق درخواست های HTTP فراهم می سازد.

در این سیستم، داده ها در دو فایل اصلی نگهداری می شوند:

- **users.json**: شامل اطلاعات کاربران سیستم
- **timex.json**: شامل رکوردهای حضور و غیاب کاربران

ساختار داده کاربران (User Data Structure)

اطلاعات هر کاربر در فایل **users.json** به صورت یک شیء JSON ذخیره می شود که شامل مشخصات حساب کاربری، وضعیت فعالیت و اطلاعات شخصی است.

نمونه ای از ساختار داده کاربر در سیستم به صورت زیر است:

```
{  
  
  "id": "10",  
  
  "viewState": 1,  
  
  "username": "l1stone9",  
  
  "email": "l1stone9@state.tx.us",  
  
  "isActive": true,  
  
  "position": "VP Marketing",  
  
  "password": "aX3!8Pm3yC5P7D,"
```

```
" personalInfo} {"
"  first_name": "Lane,"
"  last_name": "Listone,"
"  gender": true,
"  age": 34,
"  isAdmin": false
}
```

در این ساختار:

- **id** : شناسه یکتای کاربر در سیستم
- **viewState** : وضعیت نمایش یا وضعیت منطقی رکورد در سیستم
- **username** : نام کاربری برای ورود به سیستم
- **email** : ایمیل کاربر
- **isActive** : مشخص می‌کند که حساب کاربر فعال است یا خیر
- **position** : سمت یا جایگاه شغلی کاربر در سازمان
- **password** : رمز عبور کاربر برای ورود به سیستم
- **personalInfo** : شامل اطلاعات شخصی کاربر

و در بخش **personalInfo**:

- **first_name / last_name** : نام و نام خانوادگی کاربر
- **gender** : جنسیت کاربر (true یا false)
- **age** : سن کاربر
- **isAdmin** : مشخص‌کننده نقش کاربر در سیستم (ادمین یا کاربر عادی)

نقش ساختار داده در کنترل دسترسی

فیلد **isAdmin** در ساختار داده کاربران نقش مهمی در کنترل سطح دسترسی (**Access Control**) دارد.

- اگر مقدار آن **true** باشد، کاربر به عنوان **Administrator** شناخته می‌شود و به بخش **Users Management** و مدیریت کاربران دسترسی خواهد داشت.
- اگر مقدار آن **false** باشد، کاربر به عنوان **User** عادی در نظر گرفته می‌شود و فقط می‌تواند **Dashboard** و اطلاعات حضور و غیاب خود را مشاهده کند.

بخش پنجم:

1. مدیریت شاخه‌بندی پروژه (Branch Management)

برای مدیریت نسخه‌ها و همکاری تیمی در توسعه پروژه، از سیستم کنترل نسخه **Git** استفاده شده است. به منظور جلوگیری از تداخل تغییرات اعضای تیم و حفظ پایداری نسخه اصلی پروژه، از یک استراتژی **Feature Branch Workflow** استفاده شد. در این روش، یک شاخه اصلی با نام **main** به عنوان نسخه پایدار و نهایی پروژه در نظر گرفته شد. این شاخه همیشه شامل کدی است که بررسی شده و آماده اجرا است.

فرآیند توسعه به این صورت انجام شد:

1. ابتدا شاخه **main** به عنوان شاخه اصلی مخزن پروژه ایجاد شد.
2. هر عضو تیم برای پیاده‌سازی یک قابلیت جدید یا اعمال تغییرات، یک شاخه جداگانه (**Feature Branch**) از شاخه **main** ایجاد می‌کرد.
3. توسعه و تغییرات مربوط به آن قابلیت فقط در همان شاخه انجام می‌شد تا از ایجاد تداخل در کد سایر اعضای تیم جلوگیری شود.
4. پس از تکمیل تغییرات، عضو تیم یک **Pull Request** برای ادغام (**Merge**) شاخه خود با شاخه **main** ارسال می‌کرد.
5. سرگروه پروژه کدهای ارسالی را بررسی می‌کرد و در صورت تأیید، تغییرات با شاخه **main** ادغام (**Merge**) می‌شد.

2. خلاصه وضعیت اسپرینت ها

اسپرینت اول (هفته 1 و 2- پایان 18 اردیبهشت)

- 1) تحلیل اولیه نیازمندی ها
- 2) طراحی UI/UX
- 3) ساخت داشبورد اولیه
- 4) پیاده سازی واکنش گرایی (Mobile view/Desktop View)

اسپرینت دوم (هفته 3 و 4- پایان 1 خرداد)

- 1) ساخت صفحه users
- 2) نمایش لیست کاربران با استفاده از Json Server
- 3) امکان انتخاب کاربران
- 4) پیاده سازی قابلیت حذف کاربران

اسپرینت سوم (هفته 5 و 6- پایان 1 خرداد)

- 1) تکمیل قابلیت های فیلترگذاری
- 2) افزودن امکانات مرتب سازی کاربران
- 3) بهبود UI لیست کاربران

اسپرینت چهارم (هفته 7 و 8- پایان 29 خرداد)

- 1) تکمیل صفحه ایجاد کاربران
- 2) بررسی نهایی UI/UX و اصلاحات
- 3) تست کامل پروژه و رفع باگها
- 4) تکمیل و بازبینی مستندات پروژه

بخش ششم: هویت بصری سیستم (Visual Identity)

هویت بصری سیستم Codiva به گونه‌ای طراحی شده است که رابط کاربری ساده، خوانا و یکپارچه‌ای برای کاربران فراهم کند. در پیاده‌سازی رابط کاربری از TailwindCSS استفاده شده است که امکان طراحی سریع، منظم و قابل نگهداری را فراهم می‌کند. در این سیستم تلاش شده است تا با استفاده از رنگ‌بندی محدود، تایپوگرافی یکنواخت و کامپوننت‌های قابل استفاده مجدد، یک تجربه کاربری منسجم ایجاد شود.

۱. قالب‌بندی و پس‌زمینه صفحات (Background & Layout)

ساختار کلی صفحات بر اساس یک Dashboard Layout طراحی شده است که شامل یک Sidebar ثابت و بخش محتوای اصلی است.

ویژگی‌های اصلی قالب‌بندی:

- استفاده از Sidebar ثابت در سمت چپ صفحه برای ناوبری بین بخش‌های سیستم
- استفاده از پس‌زمینه تیره (Dark Sidebar) برای ایجاد تمرکز روی منوی ناوبری
- استفاده از رنگ خاکستری تیره (gray-900) برای نوار کناری
- استفاده از رنگ خاکستری تیره‌تر (gray-800) برای نمایش آیتم فعال در منو
- استفاده از ساختار Flex Layout برای مدیریت چینش عناصر

این ساختار باعث می‌شود کاربران به راحتی بین بخش‌های مختلف سیستم مانند Users Management و Dashboard جابجا شوند.

۲. پالت رنگی (Color Palette)

در طراحی رابط کاربری از رنگ‌های خنثی و حرفه‌ای استفاده شده است تا خوانایی بالا و ظاهر سازمانی ایجاد شود. رنگ‌های اصلی مورد استفاده:

- Gray-900 → رنگ اصلی پس‌زمینه Sidebar
- Gray-800 → رنگ آیتم فعال در منو
- White → رنگ متن‌ها و عناصر اصلی
- Gray-400 / Gray-300 → رنگ placeholder و مرز فیلدها

۳. تایپوگرافی (Typography)

در سیستم از تایپوگرافی ساده و خوانا استفاده شده است تا اطلاعات برای کاربران به راحتی قابل مشاهده باشد.

ویژگی‌های تایپوگرافی:

- استفاده از **font-semibold** برای تأکید بر عناصر مهم مانند دکمه‌ها و منوها
- استفاده از **text-sm** برای متن‌های رابط کاربری
- استفاده از **leading-6** برای فاصله مناسب خطوط
- استفاده از رنگ **text-white** برای نمایش متن در محیط تیره

۴. طراحی دکمه‌ها (Buttons)

در سیستم یک کامپوننت دکمه قابل استفاده مجدد طراحی شده است تا در تمام بخش‌های برنامه از یک الگوی طراحی واحد استفاده شود.

ویژگی‌های طراحی دکمه‌ها:

- گوشه‌های گرد (**rounded-md**) برای ظاهر مدرن‌تر
 - استفاده از **font-semibold** برای برجسته کردن متن دکمه
 - رنگ متن سفید (**text-white**) برای ایجاد کنتراست مناسب
 - استفاده از **shadow-sm** برای ایجاد عمق بصری
- ساختار دکمه به صورت یک **Reusable Component** پیاده‌سازی شده است تا در بخش‌های مختلف سیستم مانند ایجاد کاربر، ویرایش یا عملیات دیگر استفاده شود.

۵. فیلدهای ورودی و جستجو (Input Fields)

برای ورود اطلاعات کاربران از یک کامپوننت **Input** مشترک استفاده شده است که شامل **Label** و فیلد ورودی می‌باشد.

ویژگی‌های طراحی فیلدهای ورودی:

- استفاده از **rounded-md** برای گوشه‌های نرم

- استفاده از ring-1 و ring-gray-300 برای نمایش مرز فیلد
- استفاده از shadow-sm برای ایجاد عمق در عناصر
- نمایش متن راهنما با placeholder به رنگ gray-400

۶. یکپارچگی رابط کاربری (UI Consistency)

برای حفظ یکپارچگی طراحی در کل سیستم:

- از کامپوننت‌های مشترک (Reusable Components) مانند Button و Input استفاده شده است
- تمام استایل‌ها با استفاده از TailwindCSS Utility Classes پیاده‌سازی شده‌اند
- ساختار صفحات بر اساس یک Layout ثابت طراحی شده است

این رویکرد باعث می‌شود توسعه رابط کاربری قابل نگهداری، مقیاس‌پذیر و یکنواخت باشد.

7. لوگوی سایت:



بخش هفتم: نمودارهای UML

نمودار Use Case (نمودار موارد کاربرد)

نمودار Use Case ارائه شده، تعاملات میان بازیگران (Actors) و عملکردهای اصلی سیستم را به تصویر می کشد. این مدل بر اساس دو نقش اصلی تعریف شده است:

۱. بازیگران (Actors):

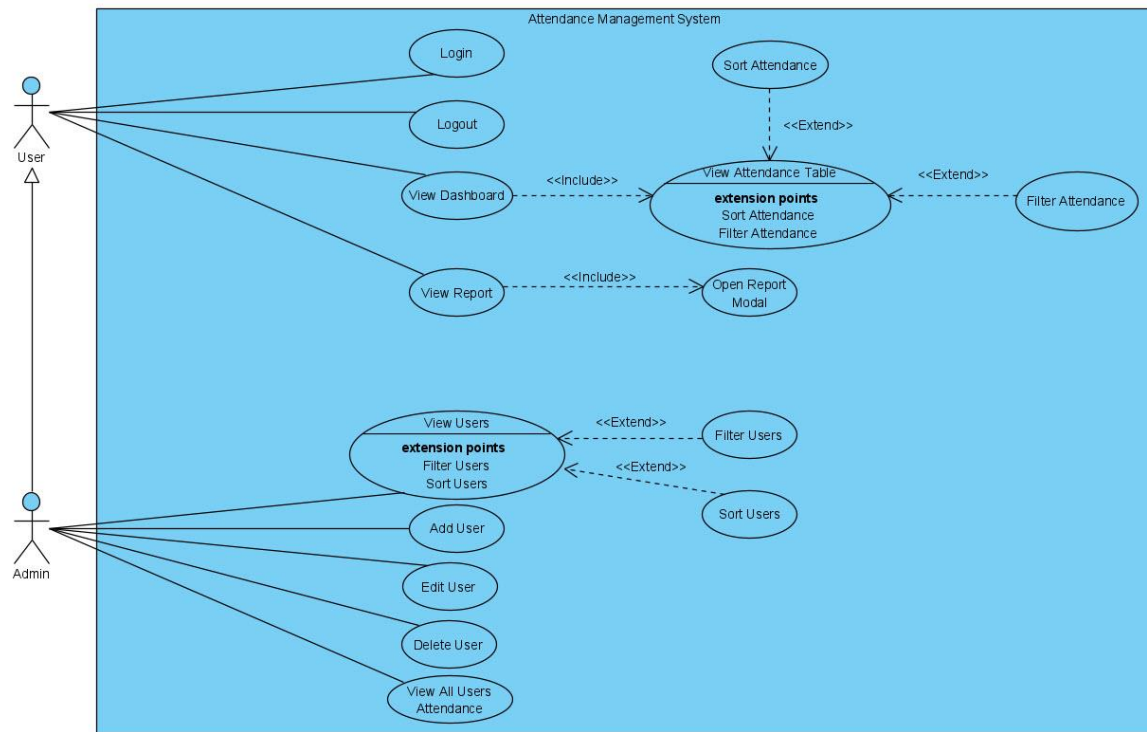
- **User** (کاربر عادی): کارمندی که به سیستم دسترسی دارد و عملیاتهای پایه مانند مشاهده داشبورد و گزارشهای فردی را انجام می دهد.
- **Admin** (مدیر): کاربری با دسترسیهای سطح بالا که علاوه بر قابلیتهای **User**، توانایی مدیریت کاربران و نظارت بر کل سیستم را داراست. (نکته: رابطه **Generalization** میان این دو، نشان دهنده ارث بری قابلیتهای کاربر توسط مدیر است).

۲. سناریوهای اصلی (Core Use Cases):

- مدیریت احراز هویت: شامل فعالیتهای **Login** و **Logout** برای امنیت سیستم.
- مدیریت داده ها (**CRUD**): این موارد کاربرد شامل افزودن (**Add**)، ویرایش (**Edit**)، حذف (**Delete**) و مشاهده کاربران است که منحصراً در اختیار مدیر سیستم قرار دارد.
- گزارش گیری و نمایش: شامل مشاهده جداول حضور و غیاب و گزارشهای اختصاصی که از طریق رابطه **<<Include>>** به عملیاتهای ضروری متصل شده اند.

۳. قابلیت های توسعه یافته (Includes & Extends):

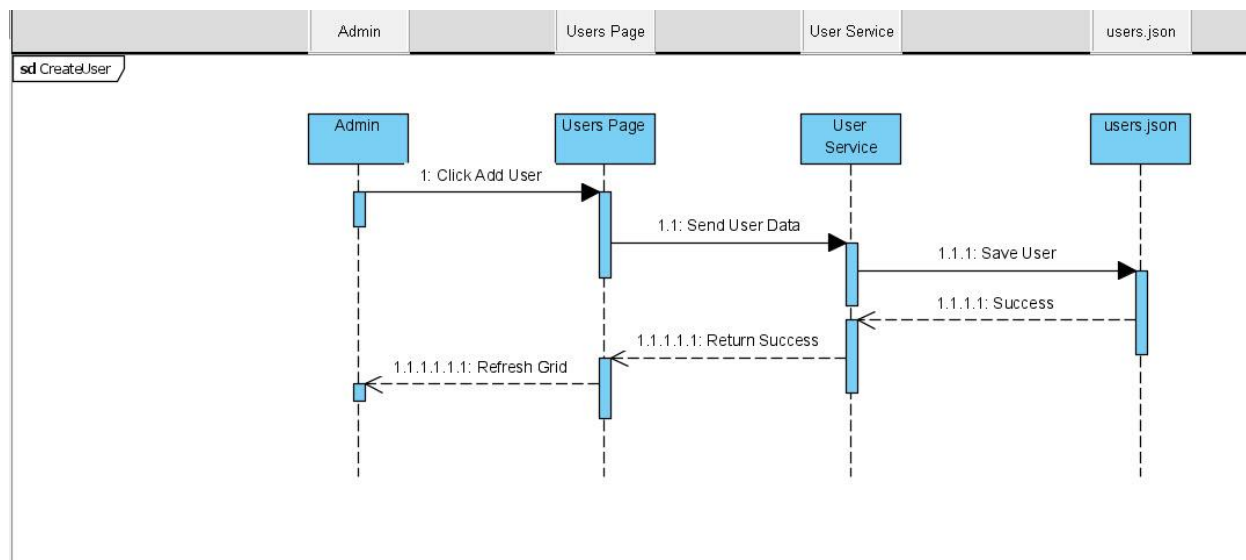
- رابطه **<<Extend>>**: برای قابلیت هایی نظیر فیلتر کردن و مرتب سازی (**Sort**) استفاده شده است؛ به این معنا که این قابلیت ها به صورت انتخابی و در صورت نیاز، به عملکردهای اصلی (مثل مشاهده جداول) اضافه می شوند.
- رابطه **<<Include>>**: برای نمایش وابستگی های ضروری در عملیاتهای مشاهده داشبورد و گزارش استفاده شده است؛ به این معنی که اجرای این **Use Case** ها بدون فرآیندهای جانبی مشخص شده، ناقص خواهد بود.



نمودارهای Sequence

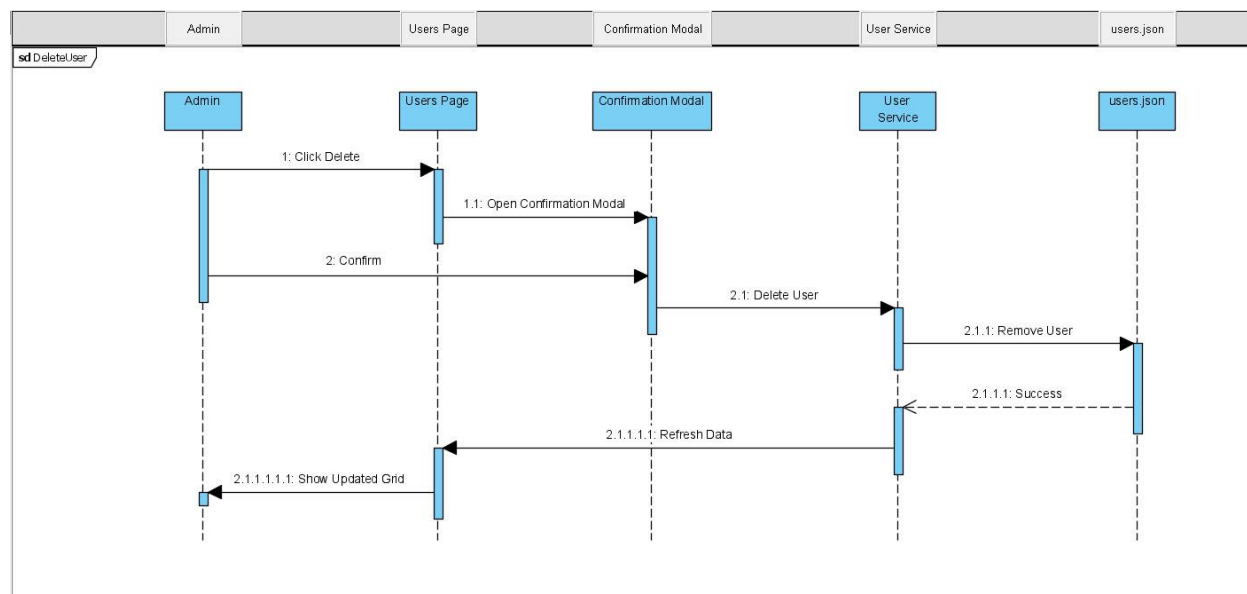
۱. نمودار توالی ایجاد کاربر جدید (Create User Sequence Diagram)

این نمودار فرآیند افزودن یک کاربر جدید توسط مدیر (Admin) را نشان می‌دهد. در این چرخه، داده‌های فرم از صفحه مدیریت به سرویس کاربر ارسال شده و پس از ذخیره‌سازی موفق در فایل users.json (پایگاه داده)، تأییدیه آن به رابط کاربری بازگشت داده می‌شود تا لیست کاربران به‌روزرسانی (Refresh) گردد.



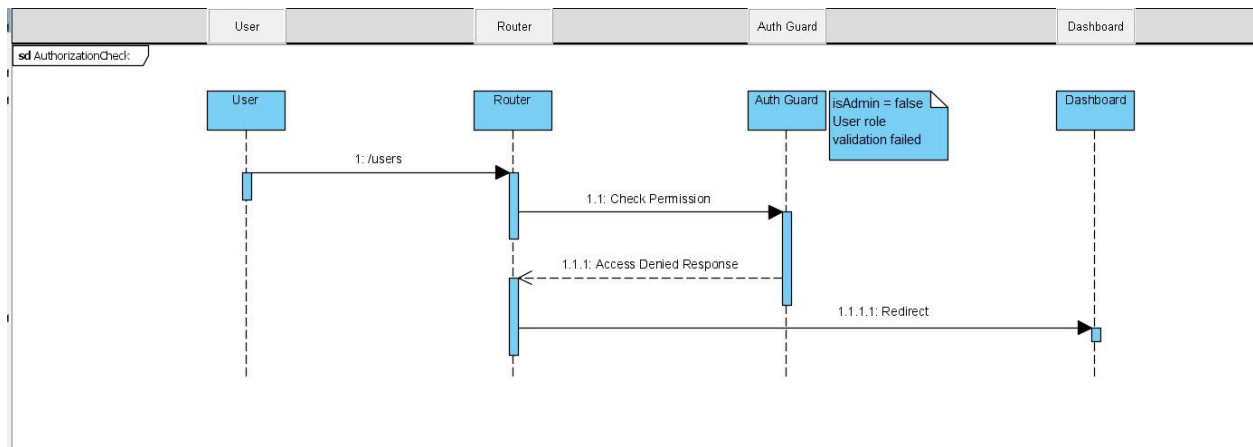
۲. نمودار توالی حذف کاربر (Delete User Sequence Diagram)

این نمودار مراحل حذف یک حساب کاربری را نمایش می‌دهد. فرآیند با درخواست حذف توسط مدیر آغاز شده و پس از نمایش و تأیید اعلان (Confirmation Modal)، دستور حذف به سرویس ارسال می‌گردد. در نهایت با حذف رکورد از فایل users.json و دریافت پاسخ موفقیت‌آمیز، جدول کاربران در صفحه اصلی به‌روزرسانی می‌شود.



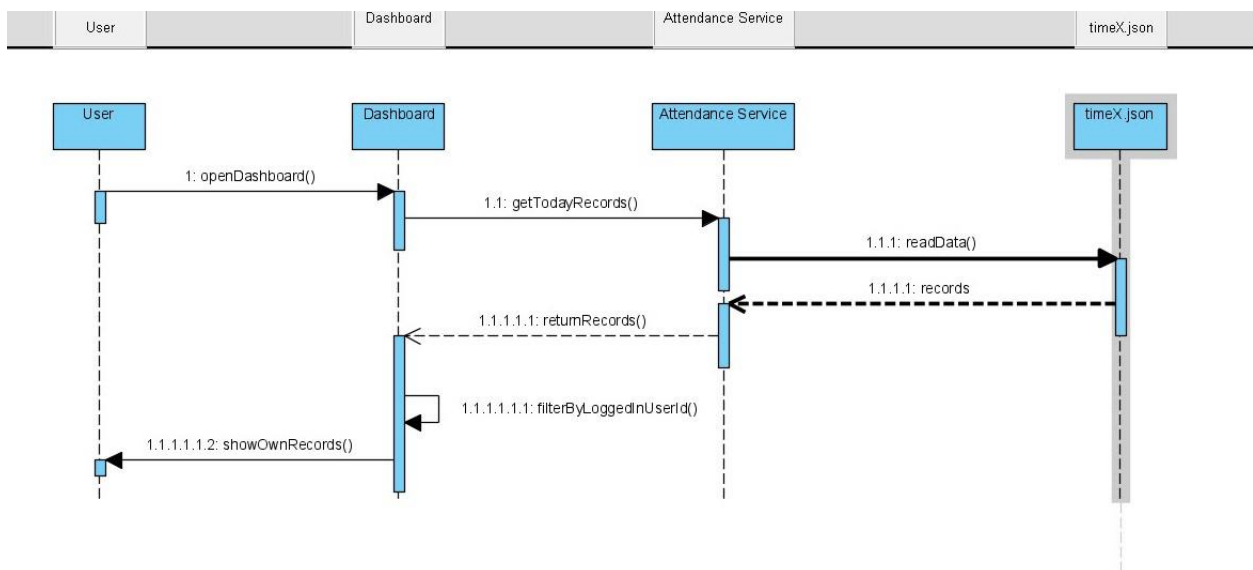
۳. نمودار توالی کنترل سطح دسترسی (Authorization Check Sequence Diagram)

این نمودار نشان‌دهنده لایه امنیتی سیستم (Auth Guard) در هنگام مسیریابی (Routing) است. زمانی که یک کاربر عادی تلاش می‌کند به مسیرهای مدیریتی (مانند /users) دسترسی پیدا کند، سیستم نقش کاربری (isAdmin) او را بررسی کرده و در صورت عدم احراز صلاحیت، دسترسی را مسدود کرده و کاربر را به صفحه داشبورد بازمی‌گرداند (Redirect).



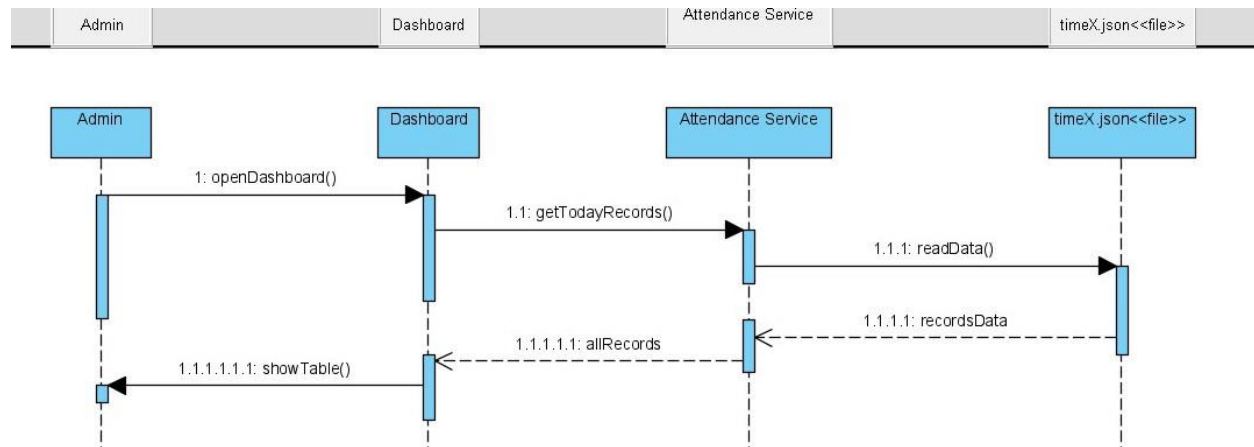
۴. نمودار بارگذاری داشبورد - کاربر عادی (Dashboard Load - Normal User)

این نمودار توالی بارگذاری اطلاعات برای یک کاربر غیر ادمین را نشان می‌دهد. سیستم پس از فراخوانی رکوردهای روز جاری از فایل timeX.json توسط سرویس حضور و غیاب، نتایج را بر اساس شناسه کاربری (UserId) فرد وارد شده فیلتر می‌کند تا کاربر فقط مجاز به مشاهده رکوردهای مربوط به خود باشد.



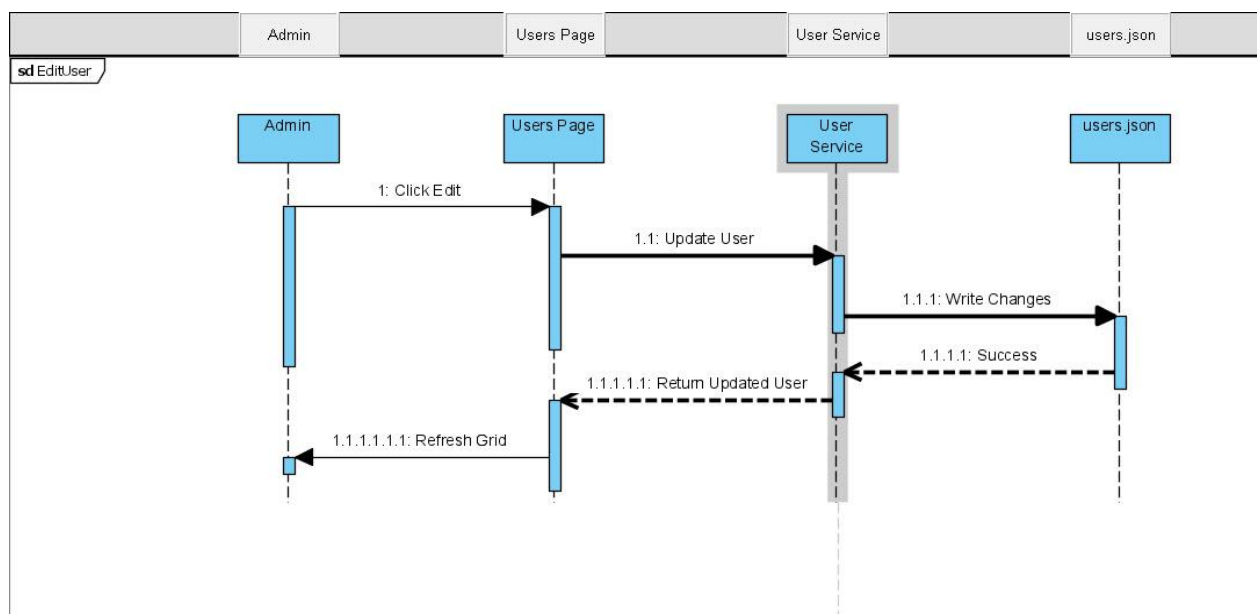
۵. نمودار بارگذاری داشبورد – مدیر (Dashboard Load - Admin)

این نمودار فرآیند نمایش اطلاعات در سطح دسترسی مدیریت را نمایش می‌دهد. برخلاف کاربر عادی، هنگامی که مدیر وارد داشبورد می‌شود، سرویس حضور و غیاب تمام رکوردهای بارگذاری شده از فایل timeX.json را بدون فیلتر محدودکننده به رابط کاربری ارسال می‌کند تا مدیر بر تمامی تردها نظارت کامل داشته باشد.



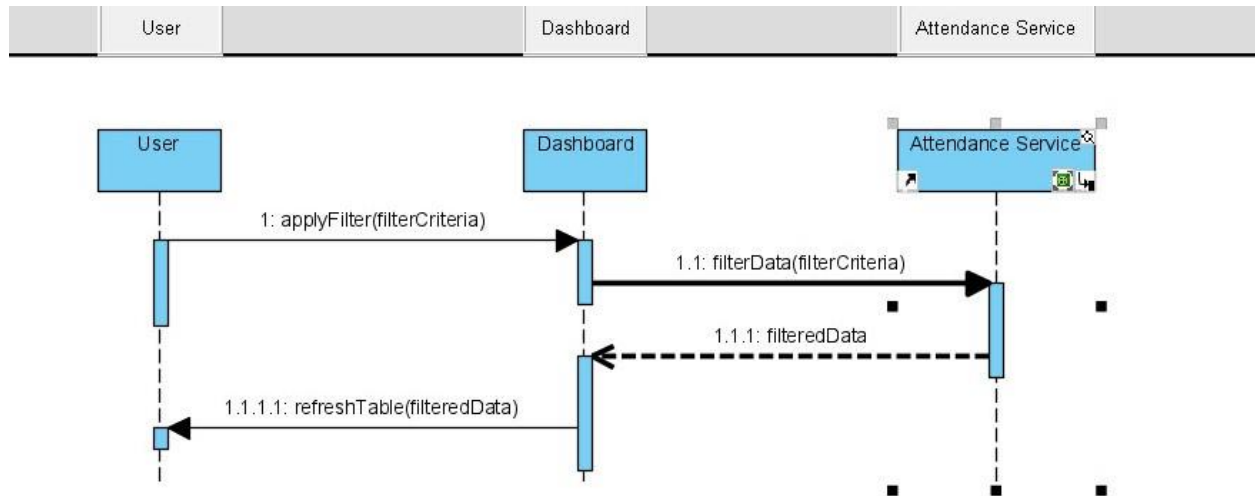
۶. نمودار توالی ویرایش اطلاعات کاربر (Edit User Sequence Diagram)

این نمودار فرآیند به‌روزرسانی اطلاعات یک کاربر موجود را نمایش می‌دهد. پس از اعمال تغییرات توسط مدیر در صفحه ویرایش، داده‌های جدید به سرویس ارسال شده و در فایل users.json بازنویسی می‌شوند. در نهایت، با دریافت پاسخ موفقیت‌آمیز، جدول کاربران با اطلاعات جدید بازخوانی و به مدیر نمایش داده می‌شود.



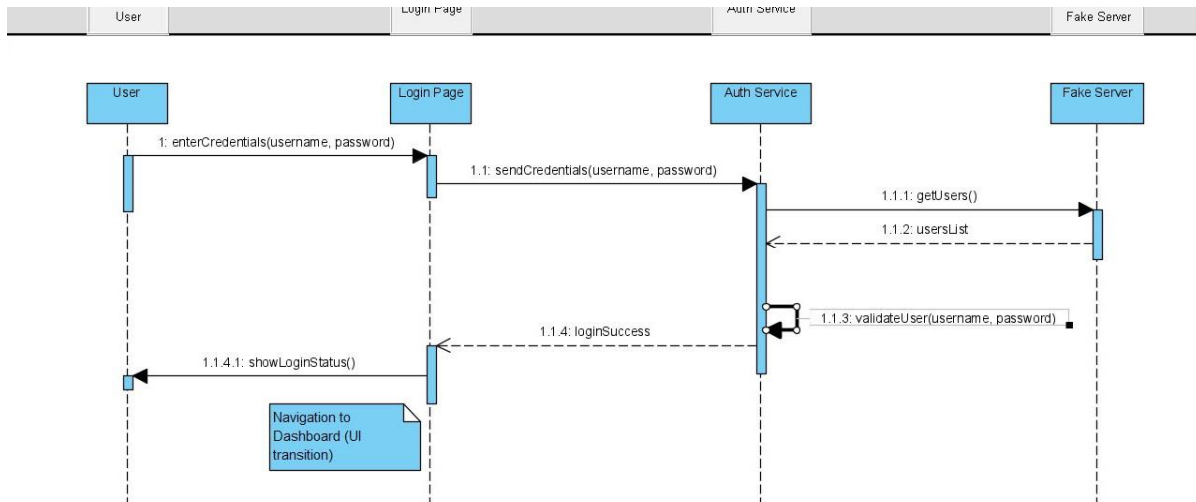
۷. نمودار توالی فیلتر اطلاعات (Filter Attendance Sequence Diagram)

این نمودار تعامل کاربر با سیستم برای محدود کردن نمایش داده‌ها (مانند فیلتر بر اساس تاریخ یا نام) را نشان می‌دهد. با اعمال معیار فیلتر در رابط کاربری، درخواست به سرویس حضور و غیاب ارسال شده و داده‌های پالایش شده (Filtered Data) به داشبورد بازمی‌گردند تا جدول حضور و غیاب بدون نیاز به بارگذاری مجدد کل صفحه، به‌روزرسانی شود.



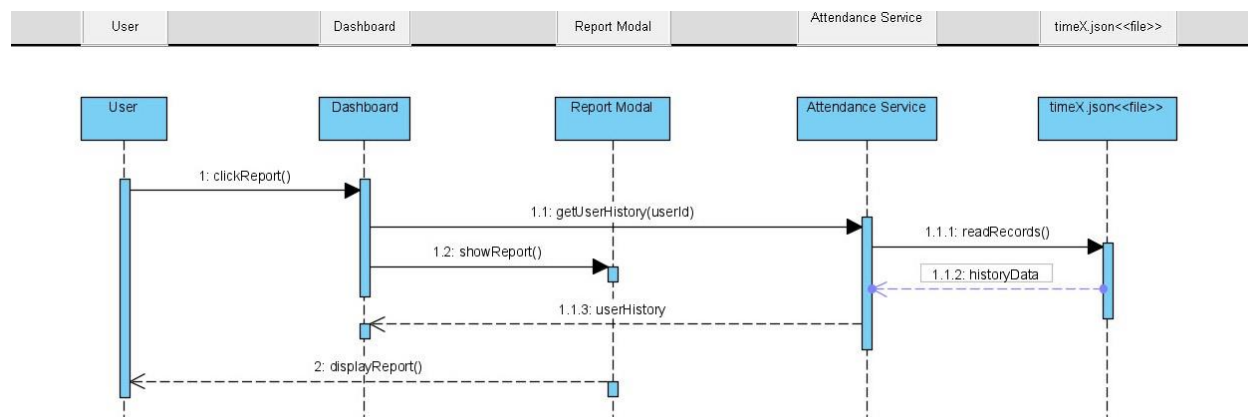
۸. نمودار توالی ورود به سیستم (Login Page Sequence Diagram)

این نمودار مراحل احراز هویت کاربر را توصیف می‌کند. پس از ورود نام کاربری و رمز عبور، سرویس Auth لیست کاربران را از Fake Server دریافت کرده و اطلاعات وارد شده را با داده‌های مرجع تطبیق می‌دهد (Validate). در صورت صحت اطلاعات، وضعیت ورود موفقیت‌آمیز به کاربر اعلام شده و وی به صفحه داشبورد هدایت (Navigate) می‌شود.



۹. نمودار توالی مشاهده گزارش کاربر (View User Report Sequence Diagram)

این نمودار فرآیند نمایش تاریخچه کامل حضور و غیاب یک کاربر در قالب Modal را نشان می‌دهد. با کلیک بر روی دکمه گزارش، سیستم شناسه کاربر (UserId) را به سرویس ارسال کرده و تمام سوابق مرتبط را از فایل timeX.json استخراج می‌کند. این اطلاعات در نهایت در پنجره گزارش (Report Modal) به صورت دسته‌بندی شده به کاربر نمایش داده می‌شود.



بخش هشتم:

1. راه اندازی و اجرای پروژه (Project Setup and Execution)

برای اجرای پروژه داشبورد مدیریت حضور و غیاب، مراحل زیر باید انجام شود:

۱. دریافت پروژه

ابتدا سورس کد پروژه از مخزن Git دریافت شده یا فایل پروژه دانلود و استخراج می شود. سپس پروژه در محیط توسعه Visual Studio Code باز می شود.

۲. نصب وابستگی ها (Dependencies)

پس از باز کردن پروژه، باید تمامی کتابخانه ها و پکیج های مورد نیاز نصب شوند. این کار با اجرای دستور `npm install` در ترمینال انجام می شود.

این دستور تمامی وابستگی های تعریف شده در فایل `package.json` را نصب می کند.

۳. اجرای سرور داده (JSON Server)

در این پروژه برای شبیه سازی بک اند و مدیریت داده ها از JSON Server استفاده شده است. این سرور داده ها را از فایل های JSON خوانده و به صورت یک REST API در اختیار برنامه قرار می دهد.

۴. اجرای برنامه

در مراحل اولیه توسعه، پروژه React با دستور زیر اجرا می شد:

`npm run dev`

اما پس از اتصال پروژه به JSON Server و تعریف اسکریپت اجرای پروژه، برنامه با دستور زیر اجرا می شود:

`npm start`

با اجرای این دستور، سرور توسعه راه اندازی شده و برنامه در مرورگر قابل مشاهده خواهد بود.

۵. مشاهده پروژه در مرورگر

پس از اجرای موفق پروژه، برنامه معمولاً در آدرس محلی (localhost) در مرورگر قابل مشاهده است و کاربران می توانند با توجه به نقش خود (مدیر یا کاربر عادی) وارد سیستم شده و از امکانات داشبورد استفاده کنند.

2. چشم‌انداز توسعه‌پذیری و ارتقای سیستم

«از منظر مهندسی نرم‌افزار، این پروژه با رعایت اصل **Separation of Concerns** (تفکیک مسئولیت‌ها) و لایه‌بندی دقیق (**Data, Service, UI**)، پتانسیل بالایی برای توسعه در آینده دارد. به دلیل استفاده از معماری سرویس‌محور و **TypeScript**، امکان جایگزینی سریع **Fake Server** با یک پایگاه‌داده واقعی (مانند **MongoDB** یا **PostgreSQL**) بدون نیاز به تغییر در منطق رابط کاربری فراهم است. همچنین، ساختار کامپوننت‌محور و استفاده از **Context API**، افزودن قابلیت‌های جدیدی همچون سیستم مرخصی، محاسبه حقوق و دستمزد یا داشبوردهای آماری پیشرفته را با کمترین هزینه زمانی و بدون ایجاد تداخل در کدهای موجود، میسر می‌سازد.»